Manual Tecnico - SIGELAB

Sistema de Gestion y Control de Laboratorios Universitarios

Curso: 017 - Programacion II
Universidad: Universidad Mariano Galvez de Guatemala
Proyecto: SIGELAB
Version: 1.0
Fecha: 29 de octubre de 2026
Integrantes: [Completar nombres y carnets del equipo]

Indice

1. Proposito del documento
2. Descripcion general
3. Tecnologias utilizadas
4. Arquitectura del sistema
5. Estructura del repositorio
6. Requisitos de instalacion
7. Configuracion y ejecucion
8. Base de datos
9. Modulos tecnicos y reglas de negocio
10. Integracion de reconocimiento facial
11. Integracion de codigo QR
12. Dashboard y reportes
13. Seguridad y configuracion sensible
14. Pruebas tecnicas y casos de aceptacion
15. Solucion de problemas
16. Mantenimiento del proyecto

---

1. Proposito del documento

Este manual explica la estructura tecnica, configuracion, instalacion y ejecucion de SIGELAB. Su objetivo es permitir que un docente o desarrollador pueda descargar el repositorio, crear la base de datos, configurar las dependencias y ejecutar la aplicacion.

SIGELAB administra laboratorios universitarios, equipos tecnologicos, prestamos, devoluciones, mantenimiento, auditoria, codigos QR, dashboard, reportes y autenticacion con validacion facial como segundo factor.

2. Descripcion general

La aplicacion principal es de escritorio y se desarrolla en Java Swing. Los datos se almacenan en Microsoft SQL Server mediante JDBC. El reconocimiento facial se ejecuta como un servicio local en Python y OpenCV/componentes equivalentes; Java consume dicho servicio durante el inicio de sesion.

El proceso principal controla los estados de un equipo:

```text
DISPONIBLE -> PRESTADO -> DISPONIBLE o DANADO
DANADO -> MANTENIMIENTO -> DISPONIBLE o BAJA
DISPONIBLE -> MANTENIMIENTO (mantenimiento preventivo)
```

3. Tecnologias utilizadas

Componente	Tecnologia
Lenguaje principal	Java 21
Interfaz de escritorio	Java Swing e IntelliJ GUI Designer (`.form`)
Gestion de dependencias	Maven
Base de datos	Microsoft SQL Server 2022 o compatible
Acceso a datos	JDBC (`mssql-jdbc`)
Seguridad de contrasenas	jBCrypt
Selector de fecha	JCalendar
QR	ZXing 3.5.3
Reconocimiento facial	Python, servicio local y camara
Control de versiones	Git y GitHub

4. Arquitectura del sistema

La arquitectura requerida y aplicada es por capas:

```text
View -> Controller -> Service -> Repository -> JDBC -> SQL Server
```

Responsabilidades:

Capa	Responsabilidad
`view`	Formularios Swing, eventos, tablas, combos y mensajes para el usuario.
`controller`	Coordinacion de acciones de interfaz.
`service`	Validaciones y reglas de negocio.
`repository`	Consultas SQL, inserciones, actualizaciones y transacciones JDBC.
`model`	Entidades del dominio: usuario, equipo, prestamo, mantenimiento, auditoria, etc.
`integration/facial`	Cliente Java que consume el servicio facial local.

Las reglas de negocio no deben depender solamente de botones ocultos, tablas filtradas o controles deshabilitados. Las operaciones criticas deben validarse desde `service` y ejecutarse de forma atomica desde `repository`.

5. Estructura del repositorio

```text
GestionyControl/
```

```
|-- pom.xml
|-- docker-compose.yaml
|-- facial-service/
|   |-- app/                  # Servicio facial Python
|   |-- sql/                  # Tablas del componente facial
|   |-- requirements.txt
|   |-- README.md
|   `-- .env.example
|-- ImagenesQR/               # Imagenes QR de ejemplo
`-- src/main/java/Proyecto/universidad/
    |-- Config/               # Conexion, script de tablas y
procedimientos SQL
    |-- controller/
    |-- enums/
    |-- integration/facial/
    |-- model/
    |-- repository/
    |-- service/
    `-- view/                 # Formularios `.java` y `.form`
```

6. Requisitos de instalacion

Instalar previamente:

1. JDK 21.
2. Apache Maven 3.9 o superior, o Maven integrado en IntelliJ IDEA.
3. Microsoft SQL Server 2022 o compatible.
4. SQL Server Management Studio (SSMS) o una herramienta equivalente.
5. Python 3.10 o superior para el componente facial.
6. Una camara web funcional para enrolamiento y validacion facial.
7. Git.

Opcionalmente se puede usar Docker Desktop para iniciar SQL Server usando `docker-compose.yaml`.

7. Configuracion y ejecucion

7.1 Descargar el proyecto

```powershell
git clone https://github.com/Daniel-codeCreator/GestionyControl.git
cd GestionyControl
```

Si ya existe una copia local vinculada al repositorio:

```powershell
git pull origin main
```

7.2 Crear la base de datos

1. Abrir SQL Server Management Studio.

2. Ejecutar `src/main/java/Proyecto/universidad/Config/tablas.sql`.
3. Ejecutar `src/main/java/Proyecto/universidad/Config/SP.sql`.
4. Ejecutar `facial-service/sql/001_create_facial_tables.sql` cuando se habilite el componente facial.
5. Verificar que existan las tablas: `Rol`, `Usuario`, `Laboratorio`, `TipoEquipo`, `Equipo`, `Prestamo`, `DetallePrestamo`, `Devolucion`, `Mantenimiento` y `Auditoria`.

El script inicial crea la base `PROYCPROGRA2`. Si se usa otro nombre, debe configurarse la conexion de Java y la configuracion del servicio facial para que ambas apunten a la base correcta.

7.3 Configurar la conexion Java

La clase `Config/connection.java` permite configurar la conexion mediante propiedades de Java o variables de entorno:

Variable	Uso
`DB_URL`	Cadena JDBC de SQL Server.
`DB_USER`	Usuario de SQL Server.
`DB_PASSWORD`	Contrasena de SQL Server.

Ejemplo en PowerShell, solo para la sesion actual:

```powershell
$env:DB_URL =
"jdbc:sqlserver://localhost:1433;databaseName=PROYCPROGRA2;encrypt=true;t
rustServerCertificate=true;"
$env:DB_USER = "sa"
$env:DB_PASSWORD = "[CONTRASENA_LOCAL]"
```

No publicar contrasenas, llaves de cifrado ni archivos `.env` con valores reales en Git.

7.4 Compilar y ejecutar Java

Con Maven:

```powershell
mvn clean compile
mvn exec:java -Dexec.mainClass="Proyecto.universidad.main"
```

Si el IDE administra la ejecucion, abrir el proyecto Maven en IntelliJ IDEA, configurar JDK 21 y ejecutar la clase `Proyecto.universidad.main`.

> Nota: si no se usa `exec-maven-plugin`, la aplicacion puede iniciarse directamente desde IntelliJ. Antes de la entrega se recomienda agregar instrucciones exactas y probarlas desde una clonacion limpia.

7.5 Iniciar SQL Server mediante Docker (opcional)

Definir una contrasena segura para SQL Server y ejecutar:

```powershell
$env:MSSQL_SA_PASSWORD = "[CONTRASENA_SEGURA]"
docker compose up -d
```

Luego ejecutar los scripts SQL descritos en la seccion 7.2.

8. Base de datos

8.1 Entidades principales

Tabla	Funcion
`Rol`	Clasifica usuarios como ADMIN, ENCARGADO o DOCENTE.
`Usuario`	Credenciales, datos personales, rol y estado.
`Laboratorio`	Ubicacion logica de equipos.
`TipoEquipo`	Catalogo de categorias de equipo.
`Equipo`	Activo tecnologico y su estado actual.
`Prestamo`	Encabezado de la entrega temporal.
`DetallePrestamo`	Equipos asociados a un prestamo.
`Devolucion`	Registro de condicion de devolucion de cada detalle.
`Mantenimiento`	Historial de intervenciones de cada equipo.
`Auditoria`	Trazabilidad de eventos relevantes.

8.2 Relaciones principales

```text
Laboratorio 1 --- N Equipo
TipoEquipo   1 --- N Equipo
Prestamo     1 --- N DetallePrestamo
Equipo       1 --- N DetallePrestamo
Equipo       1 --- N Mantenimiento
```

El script utiliza claves primarias, foraneas, campos obligatorios, restricciones `UNIQUE` y `CHECK` para proteger la integridad de los datos.

9. Modulos tecnicos y reglas de negocio

9.1 Inicio de sesion

Flujo:

```text
Usuario + contrasena -> usuario ACTIVO -> validacion facial -> acceso o denegacion
```

La contrasena se verifica con BCrypt. El acceso solo se completa si el servicio facial confirma la coincidencia. Los eventos exitosos y fallidos quedan registrados en auditoria.

9.2 Catalogos y equipos

- Laboratorio: el codigo debe ser unico.
- Equipo: el codigo de inventario debe ser unico.
- Todo equipo pertenece a un laboratorio y a un tipo de equipo.
- Estados permitidos: `DISPONIBLE`, `PRESTADO`, `DANADO`, `MANTENIMIENTO` y `BAJA`.

9.3 Prestamos

- Un prestamo debe incluir uno o varios equipos mediante `DetallePrestamo`.
- Un equipo no puede repetirse dentro del mismo prestamo.
- Solo un equipo `DISPONIBLE` puede prestarse.
- Al confirmar, los equipos cambian a `PRESTADO`.
- La operacion debe ejecutarse como transaccion: encabezado, detalles, cambios de estado y auditoria.

9.4 Devoluciones

- Solo se devuelve un detalle pendiente de un prestamo activo.
- No se puede devolver dos veces el mismo equipo.
- `BUENO` cambia el equipo a `DISPONIBLE`.
- `DANADO` cambia el equipo a `DANADO`.
- Si quedan equipos pendientes, el prestamo continua `ACTIVO`.
- Si ya no quedan pendientes, el prestamo pasa a `FINALIZADO`.

9.5 Mantenimiento

- `DANADO -> MANTENIMIENTO` para mantenimiento correctivo.
- `DISPONIBLE -> MANTENIMIENTO` para mantenimiento preventivo.
- `MANTENIMIENTO -> DISPONIBLE` si el resultado es `REPARADO`.
- `MANTENIMIENTO -> BAJA` si el resultado es `NO_REPARABLE`.
- El ingreso y cierre de mantenimiento deben registrar auditoria y ejecutarse en transaccion.

9.6 Auditoria

Eventos esperados:

```text
LOGIN_EXITOSO
LOGIN_FALLIDO
PRESTAMO_CREADO
EQUIPO_DEVUELTO
EQUIPO_ENVIADO_MANTENIMIENTO
MANTENIMIENTO_FINALIZADO
USUARIO_DESACTIVADO
```

10. Integracion de reconocimiento facial

El servicio facial se encuentra en `facial-service/`. Sus instrucciones especificas estan en `facial-service/README.md`.

Proceso general:

1. Copiar `.env.example` a `.env` dentro de `facial-service`.
2. Configurar host, puerto, base de datos, usuario, contrasena y llave de cifrado.
3. Crear un entorno virtual de Python.
4. Instalar las dependencias con `pip install -r requirements.txt`.
5. Ejecutar el script de inicio correspondiente al sistema operativo.
6. Registrar el rostro de cada usuario desde la aplicacion Java.
7. En el login, Java solicita la verificacion al servicio local antes de abrir el menu.

El reconocimiento facial es un segundo factor academico; no reemplaza la validacion de usuario, contrasena ni estado activo.

11. Integracion de codigo QR

El modulo QR usa ZXing. Cada QR representa exclusivamente el codigo de inventario del equipo.

Flujo:

```text
Imagen QR -> lectura de codigo -> consulta de equipo en SQL Server -> mostrar estado
```

La lectura QR no modifica estados por si sola. Si el equipo esta disponible puede agregarse a la seleccion del prestamo; si esta prestado, danado, en mantenimiento o de baja, debe bloquearse el prestamo.

12. Dashboard y reportes

El dashboard debe consultar la base de datos y mostrar:

- Total de equipos.
- Equipos disponibles.
- Equipos prestados.
- Equipos danados.
- Equipos en mantenimiento.
- Equipos de baja.
- Prestamos activos.
- Prestamos vencidos.

Un prestamo vencido es aquel cuya fecha esperada ya paso y conserva al menos un detalle pendiente.

Reportes minimos solicitados:

1. Equipos por estado.
2. Prestamos activos, finalizados y vencidos.

3. Prestamos entre dos fechas.
4. Historial de prestamos de un equipo.
5. Historial de mantenimiento de un equipo.

Las consultas deben estar en la capa `repository`, no directamente en las vistas Swing.

13. Seguridad y configuracion sensible

1. Usar BCrypt para almacenar contrasenas; nunca guardar contrasenas en texto plano.
2. Configurar las credenciales de SQL Server mediante variables de entorno.
3. Mantener `.env` fuera de Git; versionar solo `.env.example`.
4. No publicar llaves de cifrado facial ni contrasenas de SQL Server.
5. Validar permisos por rol en la logica del sistema, no solo ocultando botones.
6. Mostrar mensajes claros al usuario y registrar el detalle tecnico solo en consola o log controlado.

14. Pruebas tecnicas y casos de aceptacion

Antes de la entrega, documentar evidencia de los siguientes casos:

Caso	Resultado esperado
CP-01	Login con rostro correcto permite acceso.
CP-02	Login con rostro incorrecto niega acceso.
CP-03	Codigo de equipo duplicado se rechaza.
CP-04 a CP-08	Solo equipos disponibles se prestan.
CP-09 y CP-10	QR identifica y bloquea o permite segun estado.
CP-11 a CP-14	Devolucion buena, danada, sin duplicidad y parcial.
CP-15 a CP-17	Flujo completo de mantenimiento.
CP-18	Prestamo con fecha vencida y detalle pendiente aparece vencido.

Para cada caso guardar: precondicion, pasos ejecutados, resultado obtenido, resultado esperado, fecha, responsable y captura de pantalla si el docente la solicita.

15. Solucion de problemas

Problema	Revision recomendada
No conecta a SQL Server	Revisar `DB_URL`, `DB_USER`, `DB_PASSWORD`, puerto 1433, servicio SQL Server y nombre de base.
Maven no reconoce dependencias	Ejecutar `mvn clean compile` y verificar acceso a internet/repositorio Maven.
No abre el servicio facial	Revisar Python, entorno virtual, dependencias, archivo `.env`, camara y puerto configurado.
No reconoce rostro	Confirmar que el usuario fue enrolado y que la llave de cifrado no cambio.
QR no se lee	Usar imagen PNG/JPG nitida que contenga un QR valido y verificar que el codigo exista en `Equipo`.

| No aparecen datos | Confirmar que se ejecuto `tablas.sql`, existen datos de prueba y la conexion apunta a la base correcta. |

16. Mantenimiento del proyecto

1. Crear una rama para cada cambio relevante.
2. Ejecutar los casos de prueba afectados antes de confirmar cambios.
3. Usar commits descriptivos, por ejemplo:

```text
feat: agregar reporte de historial de equipo
fix: bloquear prestamo concurrente de equipo
fix: validar mantenimiento preventivo
docs: actualizar manual tecnico
```

4. Mantener sincronizados el modelo ER, scripts SQL, clases Model y reglas de negocio.
5. Antes de entregar, validar que un integrante externo pueda clonar, crear la base, configurar el facial y ejecutar el sistema siguiendo este manual.

Informacion pendiente de completar por el equipo

Antes de entregar, completar o adjuntar:

- Nombres, carnets y responsabilidades de cada integrante.
- Diagrama entidad-relacion final.
- Credenciales de prueba seguras para el docente, sin publicar contrasenas reales.
- Version exacta de SQL Server y Python usadas durante las pruebas.
- Evidencia de los 18 casos de prueba obligatorios.
- Manual de usuario separado.
- Instruccion exacta de ejecucion elegida por el equipo: IntelliJ o Maven con `exec-maven-plugin`.